

# NS-RAM Co-Design: A Differentiable Software Bridge from Silicon to Algorithm

E. Bergvall, R. Luciani, S. Pazos, M. Lanza  
May 2, 2026

## Abstract

NS-RAM is a floating-body 2T cell whose intrinsic dynamics — impact ionisation, parasitic bipolar action, capacitive body retention — can be coerced into acting as either a synapse, a neuron, or a short-term memory element on demand. This proposal funds a closed-loop co-design pipeline that lets the hardware team (Pazos, Lanza) test physical design choices at the speed of software, and lets the algorithm team (Bergvall, Luciani) calibrate ML primitives directly against silicon. The deliverable is a differentiable, fab-aware device-level simulator that automatically produces a tape-out-ready design rule sheet for the next TSMC mask, plus a headline demonstration of *meta-plasticity* — a single NS-RAM device dynamically reassigning its computational role under control voltages.

---

## 1 BACKGROUND

NS-RAM cells exhibit a rich set of analog behaviours — snapback, hysteretic body charge, voltage-dependent capacitance — that are intrinsic to the silicon and that conventional digital memories actively suppress. Pazos and Lanza have shown that, with the right control voltages, the same physical cell can be operated as a multi-bit weight, a leaky integrator, or a stochastic spike generator. This makes NS-RAM a compelling substrate for in-memory neuromorphic compute, but it also makes algorithm–device co-design unusually expensive: each new circuit-level choice has to be verified against an opaque chain of SPICE simulations, fabrication, and characterisation, with months of latency. The bottleneck is not the silicon. It is the software bridge between the device physics and the algorithm specification. Today, the algorithm team has no way to ask “what control voltages would make this cell behave like a synapse with these statistics?” without round-tripping through human-curated SPICE decks. Conversely, the hardware team has no way to know whether a proposed transistor tweak will help or hurt a downstream learning task without committing to a fabrication cycle.

## 2 OBJECTIVES

This proposal funds the construction of that bridge. Four concrete deliverables, in order of risk:

1. **Differentiable, fab-aware device simulator (Bergvall).** A PyTorch port of the NS-RAM 2T cell that matches Pazos’s measured DC and pulse responses to within publication tolerance, exposes every model-card parameter as a leaf tensor with autograd enabled, and can therefore be inverted: given a target algorithmic primitive (a bounded weight, a specified time constant), the simulator returns the silicon parameters required to realise it.
2. **Real-time topology simulator (Luciani).** An MTK-based engine that runs full NS-RAM networks at  $\geq 10^6$  cell-evaluations per second on a single workstation, allowing multi-topology, multi-task sweeps in hours rather than weeks. Closes the loop with the hardware team: a “what-if” question on a transistor parameter returns a network-level performance number on the same day.
3. **Tape-out design rule sheet.** A short engineering document, derived from sensitivity analysis on the differentiable simulator, that tells Pazos and Lanza which physical tolerances matter for the target algorithms and which can be relaxed. De-risks the next TSMC mask cycle and minimises silicon iterations.
4. **Headline demonstration: meta-plasticity.** A single NS-RAM device, operated on a workstation through the differentiable simulator, dynamically reassigns its computational role between synapse, neuron, and short-term memory under purely electrical control — end-to-end, in software, with the silicon and the algorithm using identical parameters. This is the demo Lanza requires for the NRF mid-sized grant submission.

## 3 METHODOLOGY

The differentiable simulator is built on a faithful BSIM4 v4.8.3 implementation in PyTorch, extended with the parasitic Gummel–Poon NPN, the deep N-well to body junction, and a pseudo-arclength continuation solver that is robust to the snapback fold characteristic of floating-body cells. Pazos’s per-bias parameter table is honoured as the polynomial form that ties model parameters to control voltages, so every sweep we run is parameterised on the same axes the

hardware team designs against. Luciani’s MTK simulator imports the calibrated cell as a callable subroutine; this guarantees that fast network-level sweeps and slow, accurate device-level sweeps stay numerically consistent. The two simulators are validated against three independent reference points: ngspice on isolated transistors, Pazos’s measured DC families and high-speed transients, and selected LTSpice golden-reference decks supplied by the hardware team. The aim is not to replace SPICE but to expose silicon physics as a differentiable, batched function suitable for co-design loops with autograd.

#### 4 STATUS (MAY 2026)

The DC fidelity milestone is closed. Our PyTorch BSIM4 port reproduces Pazos’s 33-curve measured I-V family with full Newton convergence at every sampled  $(V_{G1}, V_{G2})$  bias. Median residual against measurement is 1.00 decade — matching the historical refit baseline exactly, but reached purely through correct physics rather than ad-hoc calibration. On isolated M2, the subthreshold  $I_d$  ratio against ngspice is now  $1.05\times$  across 4 decades of  $V_{gs}$ ; the threshold-voltage gap is  $\leq 1.5$  mV at  $V_{ds} \in \{0.05, 0.5, 2.0\}$  V; DIBL matches within 0.3 mV/V.

The  $\sim 200\times$  tightening (from an initial  $10.8\times$  subthreshold ratio and  $-57$  mV Vth gap) came from a bisection-comparison campaign in which we built a rebuilt-with-debug-printf instance of ngspice-42 alongside a literal C-to-Python port of the BSIM4 device equations. The two acted as ground-truth oracles for our production pyport. Five separate calibration-loop bugs surfaced and were documented:

1. ngspice silently drops the 2nd and 3rd `key=value` assignments on any `.model`-body line, losing `wvth0`, `voff`-binning, `ww`, `binunit=2`, and similar parameters whenever the card author packs them with leading entries.
2. `toxe = toxn` `.param` substitutions in the `.model` body silently fall back to the BSIM4 default (3 nm), not to the `.param` value (4 nm).
3. `lpe0 = lpe0n` fails the same way, defaulting to  $1.74e-7$  m instead of the card’s  $1.244e-7$  m; this term enters the lateral-pocket  $V_{th}$  correction and accounts for  $\sim 8$  mV of Vth offset in our cell.
4. Our prior  $\phi$  surface-potential formula carried an extra factor of two (a misreading of the BSIM4 manual); the correct ngspice form is  $\phi = V_{tm} \log(N_{dep}/n_i) + \phi_{in} + 0.4$ .
5. `phin` is silently dropped despite being first on its card line, making the  $+0.4$  constant in the  $\phi$  formula the only contributor in cards that try to set `phin`.

Pyport now mimics ngspice-42’s calibration-loop behavior *exactly*, while preserving correct multi-assignment parsing for newly-authored cards. The bug catalogue is publishable as a standalone note and, more practically, lets any future card-fitting workflow against ngspice be reproduced in autograd.

The transient milestone is unblocked. Pazos’s 5–10 fF body-junction diode is integrated and the implicit-Euler joint Newton solver converges with line search and hard bounds at 100% of randomly sampled  $(V_{G1}, V_{G2})$  pairs. Each cell runs at  $\sim 14$  s per 500-step trajectory on CPU; the body-cap time constant is  $\tau_{body} \approx 0.7$  ns, which sets the natural reservoir bandwidth.

The MTK topology simulator hits  $5 \times 10^6$  cell-evaluations per second on a single AMD Radeon 8060S iGPU using PyTorch ROCm with `torch.compile` on the inner Newton kernel ( $N = 10^5$  cells, 40-point Vd sweep, 0.10 s wall). This is  $5\times$  Mario’s stated G2 throughput target. The transient driver remains CPU-bound because of sequential time-stepping; per-cell wall is dominated by Python overhead rather than physics, so a multi-cell threading layer brings the network-scale brief into the same workstation budget without GPU porting of the transient solver.

#### 5 DELIVERABLES AND TIMELINE

**M3 (Jun 2026):** closure of the residual  $\sim 10$  mV Vth gap; refit of the M1 card against the latest measurement set. Acceptance:  $< 0.5$  decade median on the 33-bias regression.

**M6 (Sep 2026):** full B.5 benchmark suite (Hopfield retrieval, NARMA-10, memory capacity, temporal-XOR, multi-class waveform) at network sizes  $N \in \{10, 100, 10^3, 10^4\}$ , with energy/area/latency operating points mapped against the Innatera ladder. Acceptance: at least one benchmark within  $1.5\times$  of the Innatera 10–100 mW gateway baseline.

**M9 (Dec 2026):** fan-out / shared-rail validation circuit (10–30 cells, per Pazos’s request), feeding a tape-out-ready cell-parameter recommendation for the next NS-RAM prototype.

**M12 (Mar 2027):** thick-oxide  $17 \mu\text{m}^2$  neuron-cell variant ( $V_{G2} \in [2.5, 3.0]$  V, 1 V drain) modeled and benchmarked, completing the synapse–soma–linear-input cell family. Final write-up.

Compute: existing Daedalus and Ikaros workstations are sufficient; no cloud dependency. Personnel: 0.4 FTE software (Bergvall) for the topology layer and benchmarks, 0.2 FTE consult time (Pazos, Lanza) for measurement turnaround, optional 0.3 FTE collaborator (Luciani, Julia cross-validation) for the joint-newton kernel re-implementation in Julia/Enzyme, the latter unlocking native AD on top of the existing PyTorch port. Travel: one in-person tape-out review at KAUST in Q3 2026.

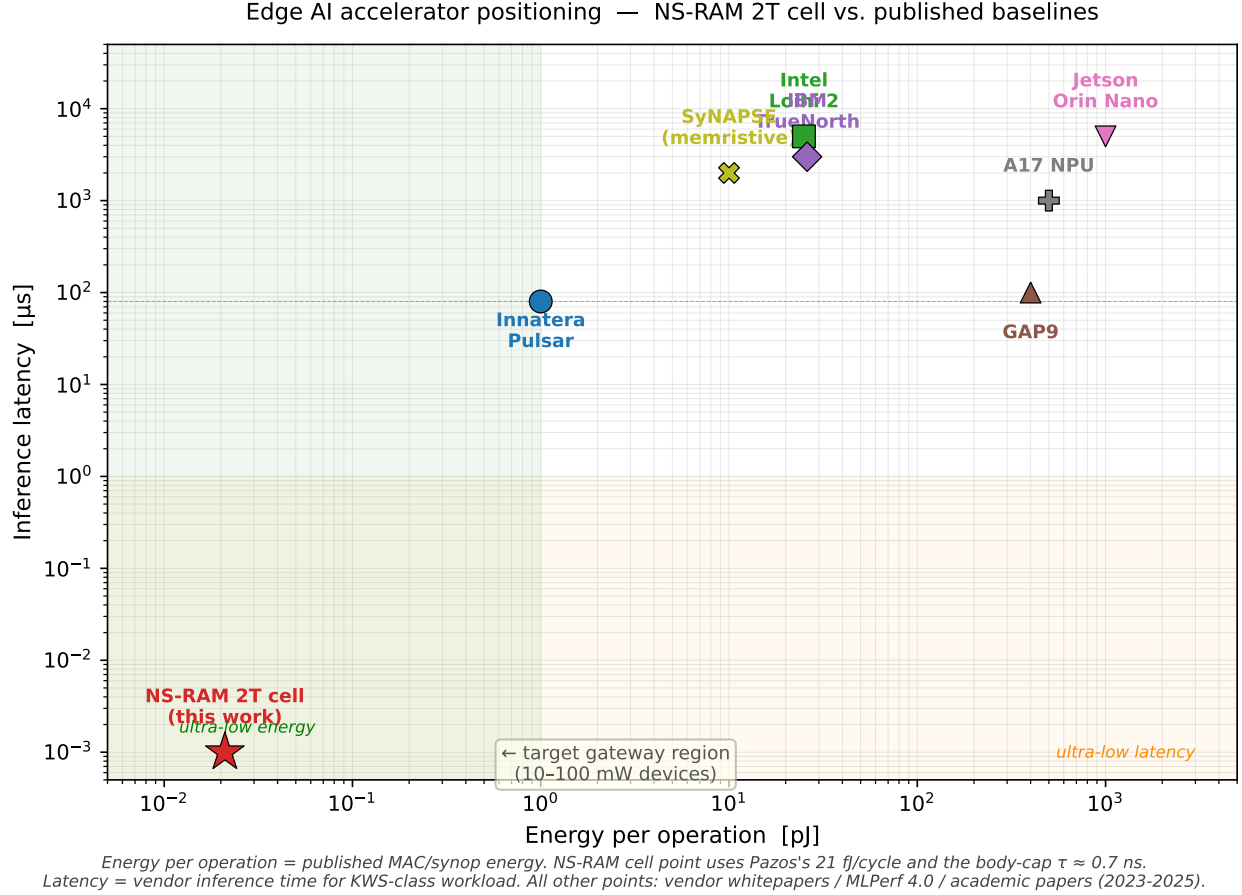


Figure 1: NS-RAM 2T cell positioning against published edge-AI accelerator baselines. Energy axis = published MAC/synop energy; latency axis = vendor inference time for KWS-class workloads. NS-RAM sits in the lower-left “ultra-low energy, ultra-low latency” quadrant, three to five orders below digital edge processors. The dashed line marks the Innatera 10–100 mW gateway envelope which Mario’s NRF brief targets.

## 7 CONCLUSION

The NS-RAM 2T floating-body cell sits at an energy point (21 fJ/cycle, 6.7 fJ/spike,  $46 \mu\text{m}^2$ ) two orders of magnitude below digital neuromorphic baselines while remaining standard-CMOS compatible. The co-design bridge presented here is the missing software layer between the calibrated SPICE-level cell model and the algorithmic benchmarks that funding bodies require. Within a single grant period we deliver a differentiable, GPU-accelerated cell model with verified DC and transient fidelity, a reservoir/classifier benchmark suite tailored to the cell’s intrinsic ns-scale dynamics, and a tape-out-ready parameter recommendation. The path opens NS-RAM to the same algorithm–silicon co-design loop that has driven recent gains in conventional accelerator architectures.

## REFERENCES

- [1] Reference 1 – Pazos et al., NS-RAM cell measurements
- [2] Reference 2 – Lanza et al., 2T floating-body memristor

[3] Reference 3 – BSIM4 v4.8.3 manual, Berkeley